Support Audit System — Developer Guide

ASP.NET Core 8 · EF Core 8 (SQL Server) · QuestPDF · Razor Pages + API Controllers

1. What this is

Digital Safe Start checklists (SHEF014) for the Stores and Dispatch support departments. It is a sibling to the Production Audit System (`github.com/SeyiEsther/TL`) and deliberately follows TL's conventions: Razor Pages only render, Controllers own every write, business logic lives in Services, and the visual language is TL's stylesheet.

The governing rule

No department, area, shift, category or task text lives in code. All of it is database rows, editable through `/Admin`.

The test: *the whole of Stores could be deleted and re-entered through Admin with no code change and no redeployment.*

This is why, for example, the checklist page decides how to render by asking the data whether its items carry a `Category` — not by checking whether the department is called "Dispatch".

2. Project layout

```
SAS.sln
src/SAS.Web/
  Models/                 Entity classes
  Data/AppDbContext.cs    DbContext + Fluent API configuration
  Migrations/             EF Core migrations (schema + seed data)
  Pages/                  GET-only renders
    Index                   Welcome card — pick date / shift / area
    Checklist               The entry screen
    Completed               Filterable history
    Admin/                  CRUD for every configuration table
  Controllers/            The writable surface
    ChecklistController     save-task, save-checkpoint, save-notes,
                            save-meta, complete
    PdfController           GET /api/checklist/pdf
  Services/               One service per concern
    ChecklistLoadService      Find-or-create a submission + its task list
    ChecklistSaveService      Every individual autosave write
    ChecklistCompletionService  Sign-off validation and completion
    HistoryListService        Completed-list query
    AdminService              CRUD for configuration tables
    PdfExportService          QuestPDF rendering
    UserService               Windows identity + AD display name
    AccessService             Roles, permissions, name resolution
    ChecklistProgress         Shared "is this answered" helpers
  wwwroot/css/site.css    Styling (TL's, copied)
  wwwroot/js/checklist.js Autosave and progress
docs/
  UserGuide.md, DeveloperGuide.md
  sql/                    Plain-SQL equivalent of every migration
```

3. Data model

Nine tables. Everything about *what* a checklist contains is configuration; everything about *what someone answered* is a submission.

Configuration

Table	Notes
`Departments`	`Name` unique
`Areas`	Unique on `(DepartmentId, Name)`
`Shifts`	Unique on `(DepartmentId, Name)`. Shifts belong to a department — they are not shared
`TaskLists`	One per `Area` + `Shift`, versioned. `IsCurrent` marks the live one
`TaskItems`	The checks. `Text`, `SortOrder`, `IsTimeBoxed`, plus five nullable category fields
`TaskCheckpoints`	The repeats on a time-boxed item — `Hr 1..Hr 12`, or `9am/11am/1pm/3pm`
`People`	HODs and Senior Operators. Unique on `(Role, DisplayName)`

Submissions

Table	Notes
`ChecklistSubmissions`	**Unique on** `(AreaId, ShiftId, ChecklistDate)` — a real database constraint, not an app check
`TaskResponses`	One per submission + item. Unique on `(ChecklistSubmissionId, TaskItemId)`
`CheckpointResponses`	One per task response + checkpoint. Unique on `(TaskResponseId, TaskCheckpointId)`

The five category fields

`Category`, `Cadence`, `ResponsibleRole`, `EscalateToRole`, `EscalationWindow` on `TaskItems`. All exist from the first migration and all are nullable.

- **Stores** leaves all five `NULL` — its tasks have no natural category and must never be forced to have one.
- **Dispatch** populates all five, seeded in the same transaction that creates the items.

These fields are not decoration. They drive behaviour — see §6.

Continuity

Resuming a checklist matches on **area + shift + date only**, never on a person's name. That is what makes a checklist a shared shift artefact rather than one person's copy, and it is enforced by the unique index rather than by convention.

---

4. How the checklist renders

`ChecklistLoadService` loads the current `TaskList` for the area and shift, then finds or creates the `ChecklistSubmission` for that date.

```
IsGridMode = items.Any(i => i.Category != null)
```

- **Grid mode** (Dispatch) — one table per category, colour-coded section rows, one column per checkpoint. Never falls back to cards, because `Category` is guaranteed populated for Dispatch from the point of seeding.
- **Card mode** (Stores) — one task per card, Done/Issue, notes revealed on Issue.

Time-boxed items

`IsTimeBoxed = true` means the item has `TaskCheckpoints`, and is answered once per checkpoint into `CheckpointResponses`. `IsTimeBoxed = false` means a single answer on the `TaskResponse` itself.

There is no "hours in shift" setting anywhere. The columns on the Dispatch grid **are** the checkpoints on its hourly items. Add a checkpoint in Admin and a column appears. This is the governing rule applied to the grid.

"Answered"

Defined once, in `ChecklistProgress.IsItemAnswered`:

- Time-boxed: **every** checkpoint has a status.
- Otherwise: the response has a status.

Progress, the completion gate and the history list all use it, so they can't drift apart.

---

5. Authentication

Copied from TL.

- `AddAuthentication(NegotiateDefaults...).AddNegotiate()`. Under IIS the identity arrives from IIS; Negotiate covers Kestrel/HTTP.sys.
- **Nothing is required to be authenticated.** An unauthenticated request falls back to the process user, so local development works off-domain. This matches TL — do not add a `FallbackPolicy` without understanding that.
- `UserService` resolves the AD display name with a one-hour `IMemoryCache` entry, and is a no-op off Windows.
- `PortalNameMatcher` is carried over from TL **verbatim** so both systems agree who someone is: exact match, "First Last" equivalence, nickname pairs (Mike/Michael), and prefix matching for account names.

Name resolution — accounts here are numbers

Windows accounts are numbers (`uk12345`), so `AccessService.CurrentDisplayNameAsync()` resolves in this order:

1. The `DisplayName` of the `People` row whose `Username` **exactly** matches the account number (case-insensitive).
2. The AD display name, if AD gave us one that isn't just the number back.
3. The raw account number.

Step 1 means it works with AD unreachable or off-domain: an admin maps the number at `/Admin/People`.

> Account matching is **exact**, deliberately. `PortalNameMatcher` does prefix matching, which would match `uk123` to `uk1234` and stamp the wrong person on an answer. Display-name matching still uses the fuzzy matcher.

---

6. Authorisation — driven by the audit data

The Dispatch audit already records who owns each check and where issues go. That data drives behaviour rather than just being printed.

Field	Effect
`ResponsibleRole = "HOD"`	Only an HOD may answer. 4 checks.
`EscalateToRole`	An Issue surfaces in the escalation panel with its window. 9 checks.
`EscalationWindow`	Shown alongside — Immediate / Same shift / Next shift / Same day.

Admin comes from configuration (`Admin:DisplayNames`, `Admin:Usernames`, `Admin:GrantAll`) exactly as in TL; HOD comes from the `People` table.

Enforcement

`ChecklistController.DenyIfNotAllowedAsync` runs on **every** write path and returns 403. `Complete` checks `CanSignOffAsync()` first.

The UI locking is a courtesy; the controller is the control. If you add a new write endpoint, it must call the guard — greying out a button is not authorisation.

---

7. Saving

Every tap is one request. There is no form post and no "save" button.

Endpoint	Purpose
`POST /api/checklist/save-task`	Done/Issue on a non-time-boxed item
`POST /api/checklist/save-checkpoint`	One checkpoint on a time-boxed item
`POST /api/checklist/save-notes`	Notes only
`POST /api/checklist/save-meta`	Auditor names, location
`POST /api/checklist/complete`	Sign-off
`GET /api/checklist/pdf`	Export

- Antiforgery via the `X-CSRF-TOKEN` header (`AddAntiforgery` header name set in `Program.cs`, matching TL). Controllers carry `[AutoValidateAntiforgeryToken]` — MVC does **not** validate automatically the way Razor Pages does.
- Notes are mandatory on an Issue, rejected server-side, not only in JS.
- The answer is stamped with the **resolved person name**, not a name typed into a box.
- A save is only reported successful once `SaveChangesAsync` returns. The UI shows "Saving…" until then and an error if it fails — an unsaved answer shows as unanswered rather than silently appearing done.

Completion validation

`ChecklistCompletionService.CompleteAsync` walks every item and refuses with a specific message if any check is unanswered, any checkpoint is unanswered, or any Issue lacks notes. It names the item and checkpoint.

---

8. Migrations and the SSMS workflow

EF Core owns the schema. **Every migration also ships as plain SQL in** `docs/sql/`, because deployment is done by hand in SSMS, not with `dotnet ef`.

Migration	What
`InitialCreate`	All nine tables, keys, unique indexes, FKs
`SeedInitialData`	Stores' four task lists transcribed from the Word documents (15/15/12/10), Consumables' empty lists, Dispatch's 24-check audit
`AddPeopleAndSplitStoresAreas`	`People` table; splits "DP1 & DP3" into DP1 + DP3; seeds the HOD list

The SQL scripts

File	When
`999_DiagnoseCurrentState.sql`	**First, always.** Read-only: migration history, row counts, current departments
`000a_CreateDatabaseAndGrantAccess.sql`	New server/database, or on error 4060. Needs sysadmin
`000_FullDeploy_IdempotentFromScratch.sql`	**The only script for a normal deploy.** Schema + seed, idempotent
`003_VerifySeedData.sql`	After deploying, to confirm
`001`, `002`, `004`	Per-migration equivalents. **Not idempotent** — reference only

> `001`/`002`/`004` use plain `CREATE TABLE` / `INSERT`. Running them on a database that already has the schema fails with "already exists" on every table. This has already happened once. Use `000`.

Regenerating after a new migration

```
cd src/SAS.Web
dotnet ef migrations add <Name> -o Migrations
dotnet ef migrations script --idempotent -o ../../docs/sql/000_FullDeploy_IdempotentFromScratch
.sql
dotnet ef migrations script <Previous> <Name>   -o ../../docs/sql/NNN_<Name>.sql
```

Then add verification queries to `003_VerifySeedData.sql`, and check the SQL parses before shipping it.

Standing rule for seed-data migrations

Any change to existing seed data must explicitly update existing rows by their stable identifying fields.

Never write a migration that only inserts rows if they don't already exist and silently skips the ones that do. That pattern is what left a previous build's rows holding old `NULL` values indefinitely while the migration reported success. Write the `UPDATE` and the `INSERT`:

```
UPDATE p SET p.SortOrder = h.SortOrder, p.IsActive = 1
  FROM People p JOIN @Hods h ON h.DisplayName = p.DisplayName
 WHERE p.Role = N'HOD';

INSERT INTO People (DisplayName, Role, SortOrder, IsActive)
SELECT h.DisplayName, N'HOD', h.SortOrder, 1 FROM @Hods h
 WHERE NOT EXISTS (SELECT 1 FROM People p WHERE p.Role = N'HOD' AND p.DisplayName = h.DisplayNa
me);
```

9. Configuration and deployment

Connection string

`ConnectionStrings:Default`. Read from `appsettings.json`, overridable by user-secrets, `ConnectionStrings__Default`, or `appsettings.Development.json` (gitignored).

`ProductionAudit` points at TL's `RittalTLSW`. **Nothing reads it yet.**

> The live connection string, password included, is currently committed in `appsettings.json` at the repo owner's explicit instruction. It is a known and deliberate exception, not an oversight. If it is ever revisited, the three options above work with no code change.

Data Protection

`Program.cs` persists Data Protection keys outside the content root, trying several writable candidates. Without this an app-pool recycle invalidates in-flight antiforgery tokens and a checklist left open fails to save with a

1 Startup logs which directory was used, or that keys are ephemeral.

IIS

Enable Windows Authentication on the site. Identity flows through to `HttpContext.User`; no extra configuration needed in the app.

Local development

```
cd src/SAS.Web
dotnet run          # uses the SAS.Web launch profile, sets ASPNETCORE_ENVIRONMENT=Development
```

Off-domain you are the process user and not an HOD. Set `"Admin": { "GrantAll": true }` in `appsettings.Development.json` to grant yourself everything — TL does the same.

---

10. Common tasks

Add a department, area or shift — Admin. No code.

Add hours to the Dispatch grid — Admin → Task lists → Edit tasks → the hourly item → add a checkpoint. A column appears.

Change task wording — Admin → Task lists → Edit tasks. For a substantial change use **New version**, which copies the list and leaves completed checklists attached to the version they were filled against.

Make someone an HOD — Admin → People & roles. If their number shows instead of their name, put the number in Account name.

Add a write endpoint — put it on a Controller, not a Page handler; call `DenyIfNotAllowedAsync`; stamp `ActorAsync`; return the error as `{ error: "..." }` so the JS surfaces it.

---

11. Decisions worth knowing

Cards vs grid is decided by the data. `Category != null`. Don't add a department name check.

Notes are per `TaskItem`, not per checkpoint or per section. TL puts one Description row at the end of each section; here the row belongs to its question, because the schema stores `Notes` on `TaskResponse`.

Changing that needs a schema change.

Historic submissions aren't rewritten. The DP1/DP3 split renamed the area but left completed checklists' recorded `Location` alone — it records what was true when it was signed.

The `Down` migration for the split refuses rather than deletes. If checklists exist against DP3 it throws instead of destroying them.

No custom auth pages. Authentication is Windows only, matching TL. There is no login form, no cookie, no user table beyond `People`.

Support Audit System — Rittal. Safe Start checklists, reference SHEF014.